

Domain 3 Study Module: Applications of Foundation Models

AIF-C01 Certification Prep Program - Task 4 of 15 Domain Weight: 28% of scored content (~14 of 50 scored questions) - the single largest domain on the exam

PART 1: DOMAIN INTRODUCTION

Why Domain 3 Carries the Most Weight

Domain 3 is the largest domain on the AIF-C01 exam, worth more than a quarter of your score. The reason is simple: this domain covers what a practitioner actually *does* day to day when working with AI. Domains 1 and 2 build the vocabulary and conceptual foundation - what AI is, what generative AI is, how foundation models work in theory. Domain 3 is where that theory becomes practice. It asks you to choose a model, write a prompt, decide whether to use RAG or fine-tuning, and judge whether a model is actually performing well. AWS designed the exam this way because the target candidate is someone who *uses* AI/ML solutions, and using a foundation model well requires exactly these skills.

How Domain 3 Builds on Domains 1 and 2

You cannot study Domain 3 in isolation. It assumes you already know:

- From Domain 1: what a model is, what training and inferencing mean, what supervised learning is, and basic performance metrics like accuracy, precision, recall, and F1 score.
- From Domain 2: what a foundation model (FM) is, what tokens and embeddings are, what a context window is, what hallucination and nondeterminism mean, and the basic shape of Amazon Bedrock as a platform.

Domain 3 takes each of those building blocks and asks: now that you know what these things are, how do you actually use them to solve a business problem?

What “Applications” Means in This Domain

In Domain 3, “applications” has two related meanings, and the exam tests both:

1. **The act of applying a foundation model** - choosing the right model, writing effective prompts, customizing model behavior, and connecting the model to outside data and tools.
2. **Software applications built on top of foundation models** - chatbots, document summarizers, search tools, and AI agents that use an FM as their reasoning engine.

When a task statement talks about “design considerations for applications that use FMs,” it is talking about both: the decisions you make when building an FM-powered tool, and the practical tradeoffs of running that tool in the real world.

Five Common Learner Misconceptions

#	Misconception	The Reality
1	“Fine-tuning is always the best way to make a model better at a task.”	Fine-tuning is often the most expensive and slowest option. Prompt engineering or RAG can solve the same business problem at a fraction of the cost and complexity in many cases.
2	“RAG changes the model itself.”	RAG does not modify the model’s weights at all. It only changes what information the model sees in its context window at the moment of the request.
3	“More examples in a prompt always produce a better answer.”	Few-shot prompting helps up to a point, but it also increases token usage (and therefore cost and latency) and can sometimes confuse the model if examples are inconsistent.
4	“A bigger model is always the better choice.”	A larger, more general model is often slower, more expensive, and not necessarily more accurate for a narrow task than a smaller, specialized model. The exam frequently rewards the “good enough and most cost-effective” answer.
5	“Evaluating a generative AI model works the same way as evaluating a traditional ML classifier.”	Generative outputs are open-ended text, image, or audio, so there is rarely one single “correct” answer. This is why GenAI evaluation relies on metrics like ROUGE, BLEU, and BERTScore, plus human judgment and LLM-as-a-judge approaches, rather than simple accuracy.

What a Non-Technical Practitioner Must Know vs. What Only Developers Need

The AIF-C01 exam does not expect you to write code, train a model, or perform mathematical analysis. For Domain 3, that distinction plays out as follows:

You ARE expected to...	You are NOT expected to...
Recognize when a scenario calls for RAG, fine-tuning, or prompt engineering	Write the code that implements RAG, fine-tuning, or a custom prompt pipeline
Identify the right Amazon Bedrock feature for a described business need	Configure Bedrock's underlying infrastructure or write API calls
Understand what ROUGE, BLEU, and BERTScore measure	Calculate ROUGE, BLEU, or BERTScore by hand
Describe the tradeoffs of different FM customization approaches	Perform the mathematical optimization involved in actually fine-tuning a model
Identify business use cases suited to foundation models	Design the data pipeline or infrastructure behind a production GenAI application

PART 2: OFFICIAL TASK STATEMENT BREAKDOWN

Task Statement 3.1: Describe design considerations for applications that use foundation models (FMs)

Plain-English Translation: Before you build anything with a foundation model, you need to think through a series of decisions: which model to use, how to tune its behavior, whether to connect it to outside data, and how to weigh the cost and complexity of different customization paths.

Specific Learner Skills: - Identify FM selection criteria: cost, modality, latency, multi-lingual support, model size, model complexity, customization options, input/output length limits, and prompt caching availability. - Explain how inference parameters, especially temperature, affect model output. - Define RAG and describe how Amazon Bedrock Knowledge Bases implements it. - Identify AWS services that store vector embeddings for RAG: Amazon OpenSearch Service, Amazon Aurora, Amazon Neptune, and Amazon RDS for PostgreSQL (pgvector). - Explain the cost and complexity tradeoffs across pre-training, fine-tuning, in-context learning, RAG, and model distillation. - Define the role of AI agents and describe their business applications.

Exam Watch-Out: The exam loves to test whether you know that RAG does NOT change model weights, while fine-tuning DOES. If a scenario says a company wants to “keep the model unchanged” but improve accuracy with company data, RAG is almost always the answer, not fine-tuning.

Three Realistic Exam Scenario Stems: 1. “A company wants its customer support chatbot to answer questions using the company’s constantly updated product manuals, without retraining the underlying model. Which approach should the company use?” 2. “A team is deciding between Amazon OpenSearch Service and Amazon Neptune to store vector embeddings for a RAG-based application. Which factor would MOST influence this decision?” 3. “Which inference parameter would a developer adjust to make a foundation model’s responses more creative and varied rather than consistent and predictable?”

Task Statement 3.2: Choose effective prompt engineering techniques

Plain-English Translation: Learn the vocabulary and techniques used to write prompts that reliably produce good output, and understand the risks that come with poorly secured or poorly designed prompts.

Specific Learner Skills: - Define prompt constructs: context, instruction, and negative prompts. - Define and differentiate prompting techniques: zero-shot, single-shot, few-shot, chain-of-thought, and prompt templates. - Identify prompt engineering best practices: improving response quality, supporting experimentation, using guardrails, specificity and concision, and using multiple instructions for complex tasks. - Define prompt-related risks: prompt exposure, prompt poisoning, prompt hijacking, and jailbreaking. - Describe prompt versioning and management using Amazon Bedrock Prompt Management.

Exam Watch-Out: Learners frequently confuse prompt hijacking (an attacker overrides the intended behavior of the prompt) with prompt poisoning (malicious instructions are injected through external data the model retrieves or processes, such as a poisoned document in a RAG pipeline). Know which term applies to which attack vector.

Three Realistic Exam Scenario Stems: 1. “A prompt engineer provides the model with two worked examples of the desired input/output format before asking it to complete a new, similar task. Which prompting technique is being used?” 2. “An attacker embeds hidden instructions inside a webpage that a RAG application later retrieves and feeds to the model as context, causing the model to ignore its original instructions. Which type of risk does this describe?” 3. “A company wants to track changes to its production prompts over time and roll back to a previous version if a new prompt underperforms. Which AWS capability supports this requirement?”

Task Statement 3.3: Describe the training and fine-tuning process for FMs

Plain-English Translation: Understand the stages a foundation model goes through, the different ways a model can be fine-tuned for a specific need, and what is required to prepare good training data.

Specific Learner Skills: - Describe pre-training, fine-tuning, continuous pre-training, and distillation. - Define fine-tuning methods: instruction tuning, domain-specific adaptation, transfer learning, and continuous pre-training. - Describe data preparation for fine-tuning: curation, governance, sizing, labeling, representativeness, and RLHF.

Exam Watch-Out: Continuous pre-training and fine-tuning are commonly confused. Continuous pre-training keeps teaching the model general knowledge using more unlabeled data from a domain (e.g., more legal text), while fine-tuning adapts the model to a specific task using labeled, often instruction-formatted, examples.

Three Realistic Exam Scenario Stems: 1. “A healthcare company wants a foundation model to become broadly more knowledgeable about medical terminology using a large volume of unlabeled clinical text, without targeting any single task. Which technique is MOST appropriate?” 2. “Which factor is MOST important when preparing a dataset for fine-tuning a foundation model to reduce the risk of biased or unbalanced outputs?” 3. “A team incorporates ratings from human reviewers directly into the model’s training process to better align outputs with human preferences. Which technique does this describe?”

Task Statement 3.4: Describe methods to evaluate FM performance

Plain-English Translation: Learn how to judge whether a foundation model, or an application built on top of one, is actually doing a good job - both from a technical metrics standpoint and a business outcomes standpoint.

Specific Learner Skills: - Determine appropriate evaluation approaches: human-in-the-loop evaluation, benchmark datasets, and Amazon Bedrock Model Evaluation. - Identify automated metrics: ROUGE, BLEU, BERTScore, and LLM-as-a-judge. - Determine whether an FM meets business objectives (productivity, engagement, task completion). - Identify evaluation approaches for FM-based applications: RAG pipelines, agents, and automated workflows. - Identify business alignment metrics: task completion rate, user satisfaction, and cost per interaction.

Exam Watch-Out: ROUGE is for summarization, BLEU is for translation. The exam frequently swaps these in distractor answers. Remember: ROUGE = Recall-Oriented (summarization, “did the summary capture the key content?”); BLEU = precision-oriented (translation, “does the translation match a reference translation closely?”).

Three Realistic Exam Scenario Stems: 1. “A company wants to automatically evaluate the quality of a foundation model’s text summaries against human-written reference summaries. Which metric is MOST appropriate?” 2. “Which AWS service allows a company to run both automatic and human-based evaluations of foundation model outputs within a managed workflow?” 3. “A company has deployed a customer service agent built on a foundation model and wants to measure whether it is meeting business goals. Which metric would be MOST useful?”

PART 3: CORE CONCEPT NOTES - DOMAIN 3 DEEP DIVE

3A. Prompt Engineering - The Practitioner’s Core Skill

Prompt engineering is the practice of designing the input text (the prompt) given to a foundation model in order to reliably produce the output you want. It is the lowest-cost, fastest way to influence model behavior, since it requires no training data and no changes to the model itself.

Zero-Shot Prompting *What it is:* Asking the model to perform a task with instructions only - no examples provided. *When to use it:* For simple, well-understood tasks where the model’s general training is already sufficient. *Plain-English example:* “Summarize this email in two sentences.” *Exam differentiator:* Zero examples. If the prompt contains no demonstration of the desired output, it is zero-shot.

Single-Shot (One-Shot) Prompting *What it is:* Providing exactly one example of the input/output pattern before asking the model to complete a new instance. *When to use it:* When the desired output format is somewhat unusual and one example clarifies it sufficiently. *Plain-English example:* “Here is a product review and its sentiment label: Review: ‘Great battery life.’ Sentiment: Positive. Now label this review: ‘The screen cracked on day one.’” *Exam differentiator:* Exactly one example given.

Few-Shot Prompting *What it is:* Providing several (typically two to five) examples of the input/output pattern before the actual request. *When to use it:* When the task has nuance that a

single example cannot fully convey, such as varied formatting or edge cases. *Plain-English example:* Giving the model three sample customer complaints, each labeled with the correct department, before asking it to classify a fourth complaint. *Exam differentiator:* Multiple examples; more token usage than zero-shot or single-shot, so there is a cost and context-window tradeoff.

Chain-of-Thought Prompting *What it is:* Instructing the model to reason through a problem step by step before producing a final answer, rather than jumping straight to a conclusion. *When to use it:* For multi-step reasoning, math-like problems, or tasks where intermediate logic matters and errors are easy to make by jumping to conclusions. *Plain-English example:* “Walk through your reasoning step by step, then provide the final answer: A store had 120 items, sold 45, and received a shipment of 30. How many items does it have now?” *Exam differentiator:* The defining phrase is reasoning shown explicitly, often cued by language like “think step by step” or “explain your reasoning.”

Role Prompting / Persona Prompting *What it is:* Assigning the model a specific role, persona, or point of view to shape the tone, vocabulary, and perspective of its response. *When to use it:* When you want consistent tone or domain framing, such as writing as a financial advisor or a friendly customer support agent. *Plain-English example:* “You are a senior financial analyst. Explain this quarterly report to a non-technical executive.”

System Prompts vs. User Prompts *What goes where:* A system prompt sets persistent, overarching instructions and constraints for the model’s behavior across an entire conversation (tone, role, rules it must always follow, things it must never do). A user prompt is the specific, individual request made within that conversation. *Why it matters:* System prompts are typically set once by the application developer and are not visible to the end user; user prompts change with every turn of conversation. Confusing the two is a common source of unpredictable behavior; for example, putting strict safety rules in a user prompt instead of the system prompt makes them easier for an end user to override.

Prompt Structure Best Practices A well-structured prompt typically includes: - **Context:** background information the model needs to understand the situation. - **Instruction:** a clear, specific statement of the task to perform. - **Output format guidance:** how the answer should be structured (a list, a table, a fixed word count, JSON, and so on). - **Negative prompting:** explicit statements of what to avoid (see below).

Negative Prompting *What it is:* Explicitly telling the model what NOT to do or include, rather than only stating what it should do. *Plain-English example:* “Summarize this article in plain language. Do not include statistics or direct quotations.” *Why it matters:* Without negative prompting, models often default to including unwanted elements (overly technical language, marketing fluff, excessive length) simply because nothing told them to avoid it.

Common Prompt Engineering Mistakes and Fixes

Mistake	Fix
Vague instructions (“make this better”)	Be specific about what “better” means: tone, length, audience, format
No output format specified	State the exact format needed (bullet list, JSON, 100-word summary)
Overloading a single prompt with too many unrelated tasks	Break the request into smaller, sequential prompts or clearly numbered instructions

Mistake	Fix
Assuming the model remembers prior unstated context	Always include necessary context directly in the prompt or conversation history
Ignoring negative prompting	Explicitly state constraints and exclusions

Comparison Table: Prompt Engineering Techniques

Technique	Definition	When to Use	Example	Exam Clue
Zero-shot	Instructions with no examples	Simple, well-known tasks	“Translate this sentence to French.”	“No examples provided”
Single-shot (one-shot)	Exactly one example given	Format needs slight clarification	One labeled example, then a new request	“One example”
Few-shot	Multiple examples given	Nuanced or pattern-based tasks	Three to five labeled examples, then a new request	“Several/multiple examples”
Chain-of-thought	Model reasons step by step before answering	Multi-step logic or math-like problems	“Think through this step by step”	“Step by step,” “show your reasoning”
Role/persona prompting	Model is assigned a role or point of view	Consistent tone or domain framing needed	“You are a tax advisor...”	“Act as,” “you are a...”
Prompt template	Reusable, structured prompt with placeholders	Repeated tasks at scale (e.g., many product descriptions)	A template with fields like {product_name}, {features}	“Reusable,” “standardized format”

3B. Retrieval-Augmented Generation (RAG)

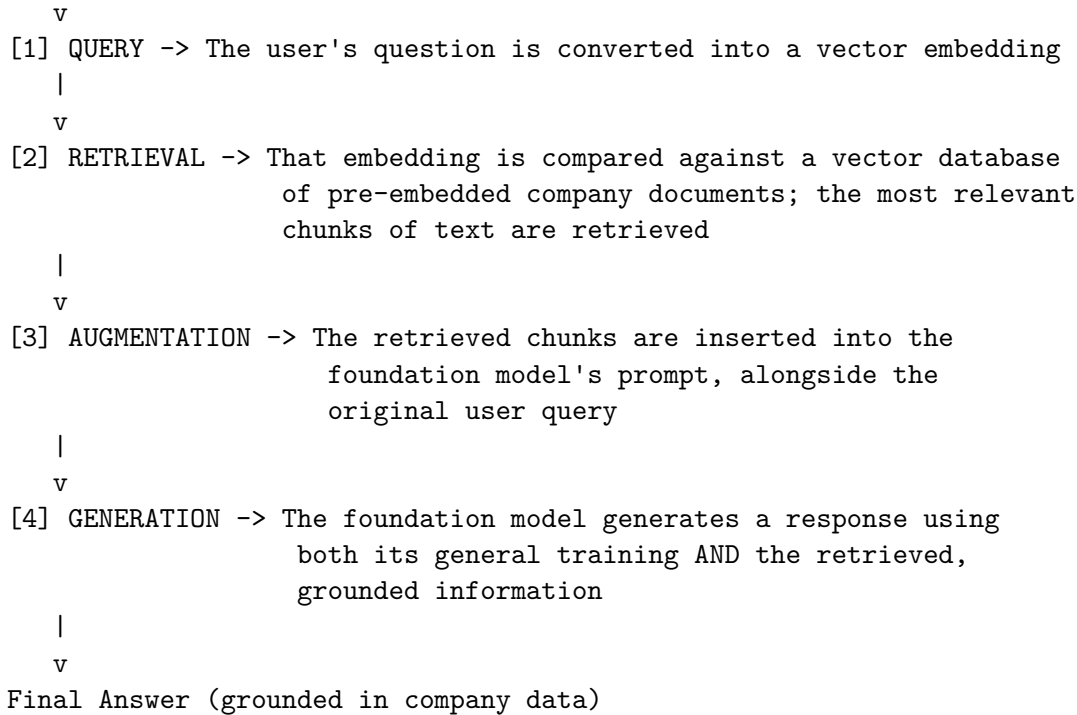
RAG in Plain English

The common analogy is that RAG turns a “closed-book exam” model into an “open-book exam” model: instead of relying only on what it memorized during training, the model is allowed to look things up before answering. A more precise way to think about it: RAG gives the model a research assistant. Before the model writes its response, a separate retrieval step searches a knowledge source for the most relevant information and hands that information to the model as part of its prompt. The model then writes its answer grounded in that retrieved material, rather than purely from memory.

The RAG Architecture (Text-Based Flow)

User Query

|



Why RAG Reduces Hallucinations

A foundation model hallucinates when it generates content that sounds plausible but is not actually true, often because it is filling gaps in its training data with a confident guess. RAG reduces this risk because the model is given verified, relevant source material at the moment it generates its answer, which anchors (grounds) the response in real, current, and traceable information rather than relying solely on patterns memorized during training.

When to Use RAG vs. Fine-Tuning vs. Prompt Engineering: A Decision Framework

Ask Yourself...	If the Answer Points To...
Does the model just need clearer instructions or examples to perform the task?	Prompt engineering
Does the model need access to current, proprietary, or frequently changing information it was not trained on?	RAG
Does the model need to learn a new skill, style, format, or specialized behavior that examples alone cannot teach?	Fine-tuning
Is cost and speed of implementation the top priority?	Prompt engineering, then RAG, then fine-tuning (in that order of cost)
Does the underlying knowledge change frequently (daily, weekly)?	RAG (fine-tuning would require constant retraining)

Embeddings in Plain English

An embedding is a numerical representation of a piece of text (a word, sentence, or document) that captures its meaning in a way a computer can compare mathematically. Two pieces of text with

similar meaning end up with embeddings that are mathematically “close” to each other, even if they use completely different words. This is what allows a RAG system to find documents that are conceptually relevant to a question, not just documents that share the exact same keywords.

Vector Databases

A vector database is a specialized data store designed to hold embeddings and perform fast similarity searches across millions of them. AI applications use vector databases because traditional databases are built to find exact matches or simple ranges, not to answer “which of these thousands of documents is most similar in meaning to this question?” AWS options for storing vectors include Amazon OpenSearch Service, Amazon Aurora, Amazon Neptune, and Amazon RDS for PostgreSQL using the pgvector extension.

Amazon Bedrock Knowledge Bases

Amazon Bedrock Knowledge Bases is the managed AWS feature that implements RAG without requiring a company to build the retrieval pipeline from scratch. It connects to a data source (commonly Amazon S3), automatically chunks the documents, generates embeddings, stores them in a connected vector database, and handles the retrieval and augmentation steps at query time. On the exam, if a scenario describes wanting to “ground” a foundation model in company documents stored in S3 without building custom retrieval infrastructure, Amazon Bedrock Knowledge Bases is almost always the intended answer.

Chunking

Chunking is the process of breaking large documents into smaller, manageable pieces before they are embedded and stored in a vector database. Chunking matters for retrieval quality because chunks that are too large may contain irrelevant information that dilutes the match, while chunks that are too small may lose important context. Well-designed chunking strategies balance these two risks.

What Affects Retrieval Quality

Practitioners can influence retrieval quality through: the chunking strategy (chunk size and overlap), the quality and organization of the source documents, the choice of embedding model, and how the retrieved chunks are ranked and filtered before being passed to the model. Poor source data or overly large chunks are common causes of irrelevant or noisy retrieval results.

3C. Amazon Bedrock - Deep Dive for Domain 3

Amazon Bedrock (Core) *Purpose:* A fully managed service that provides API access to foundation models from multiple providers (including Amazon’s own Nova models) through a single, consistent interface. *How it works conceptually:* Rather than hosting and managing your own model infrastructure, you call Bedrock’s API, specify the model you want, and receive a response. AWS manages the underlying compute. *Exam clue phrases:* “without managing infrastructure,” “access multiple model providers through a single API,” “serverless access to foundation models.”

Amazon Bedrock Knowledge Bases *Purpose:* Managed RAG. Connects an FM to a company’s own data for grounded, accurate responses. *How it works conceptually:* Ingests documents (often from Amazon S3), automatically chunks them, creates embeddings, stores them in a vector store,

and performs retrieval and augmentation at query time. *Exam clue phrases:* “ground responses in company data,” “connect to S3 documents,” “without retraining the model.”

Amazon Bedrock Agents *Purpose:* Enables a foundation model to take autonomous, multi-step actions by calling external tools and APIs to complete a task, rather than just answering a single question. *How it works conceptually:* The agent breaks a complex user request into a sequence of steps, decides which tools or APIs to call at each step (orchestration), executes those calls, and synthesizes the results into a final response. This is the practical building block for “agentic AI” introduced conceptually in Domain 2. *Exam clue phrases:* “complete a multi-step task,” “call an API to look up order status,” “autonomously take action,” “orchestrate a sequence of steps.”

Amazon Bedrock Guardrails *Purpose:* Sets safety and policy limits on what a foundation model can say or do. *How it works conceptually:* Applies configurable content filters (e.g., blocking violent or hateful content), topic restrictions (denying certain subjects entirely), PII masking/redaction, word filters, and grounding checks that verify a response is supported by retrieved source material. *Exam clue phrases:* “prevent the model from discussing competitor products,” “redact personal information from responses,” “filter harmful content,” “verify the response is grounded in the source documents.”

Amazon Bedrock Model Evaluation *Purpose:* A managed service for assessing how well a foundation model performs on a given task. *How it works conceptually:* Supports automatic evaluation (using metrics like accuracy, robustness, and toxicity, and reference-based metrics such as ROUGE and BERTScore where relevant) as well as human evaluation workflows where reviewers score model outputs against custom criteria. *Exam clue phrases:* “compare the performance of two foundation models,” “run a managed evaluation without building custom tooling,” “incorporate human reviewers into model evaluation.”

Amazon Bedrock Fine-Tuning *Purpose:* Allows a company to customize a foundation model’s behavior using its own labeled examples. *How it works conceptually:* The customer supplies a labeled dataset; Bedrock manages the underlying fine-tuning job and produces a customized version of the model that the customer can then use for inference. This is distinct from continued pre-training, which uses large volumes of unlabeled domain text rather than labeled task examples. *Exam clue phrases:* “train the model on labeled examples specific to our business,” “create a custom model based on our own data.”

Amazon Bedrock Model Customization: The Full Spectrum Bedrock supports a spectrum of customization, generally increasing in cost and complexity from left to right: Prompt engineering -> RAG (Knowledge Bases) -> Fine-tuning -> Continued pre-training -> Training a model from scratch (out of scope for typical Bedrock usage, far more costly).

3D. Model Selection - How Practitioners Choose a Foundation Model

Selection Criteria

Criterion	What It Means	Why It Matters
Cost	Price per token (input and output), and pricing model (on-demand vs. provisioned throughput)	Directly affects ongoing operating expense, especially at scale
Capability	How well the model performs on the specific task (writing, coding, reasoning, summarization)	A model strong in one area may be weak in another
Latency	How quickly the model returns a response	Critical for real-time applications like chatbots
Context window	How much text/data the model can consider at once	Larger windows support longer documents or conversation history, but often cost more
Modality	Whether the model handles text only, images, audio, or multiple modalities	Must match the application's input/output needs
Multi-lingual support	Whether the model performs well in the languages needed	Critical for global or non-English-first applications
Model size and complexity	Larger models are often more capable but slower and costlier	Bigger is not always better for narrow tasks
Customization options	Whether the model supports fine-tuning, continued pre-training, etc.	Determines how far the model's behavior can be adapted
Prompt caching availability	Whether repeated portions of prompts can be cached to reduce cost/latency	Useful for applications that reuse large context repeatedly

Model Benchmarks and Leaderboards

Benchmarks and leaderboards rank foundation models against standardized tasks and datasets. They are useful for a general sense of relative capability, but they have real limitations: benchmark performance does not always predict real-world performance on a company's specific use case, and benchmarks can become outdated or be "gamed" by models trained with awareness of the benchmark questions.

When a Smaller, Specialized Model Beats a Large General Model

A smaller model that has been fine-tuned or is purpose-built for a narrow task often outperforms a much larger general-purpose model on that specific task, while also being faster and cheaper to run. This is common in classification tasks, structured data extraction, or any narrowly defined repeated task.

The "Good Enough" Principle

For most business use cases, the goal is not to use the most powerful model available, but the most cost-effective model that reliably meets the business requirement. The exam frequently rewards answers that reflect this principle over answers that simply pick the biggest or newest model.

Exam Scenario Pattern

“A company needs to [perform some specific, narrow task] at high volume and low cost, with response times under [X] seconds. The MOST appropriate model choice is...” -> look for the smaller, faster, more cost-effective model that still meets the stated requirement, not the largest or most general-purpose option.

Task-Specific Model Considerations

Task	Key Considerations
Text generation	Creativity vs. consistency (temperature), tone control
Summarization	Context window size to fit long source documents
Question answering (Q&A)	Grounding ability, accuracy, hallucination risk
Code generation	Specialized training on code, support for relevant programming languages
Embeddings	Embedding model quality and dimensionality, not generative quality
Image generation	Modality support, style controllability, resolution/output limits

3E. Foundation Model Customization - The Full Spectrum

Approach	Training Data Needed?	Cost	Speed of Implementation	Best For
Prompt engineering	None	Lowest	Immediate	Quick wins, simple tasks, experimentation
RAG	None (uses existing documents)	Low to moderate	Fast	Grounding in current or proprietary data, reducing hallucination
Fine-tuning	Labeled, task-specific examples	Moderate to high	Slower	Teaching a new skill, style, or specialized task behavior
Continued pre-training	Large volume of unlabeled domain text	High	Slowest	Building broad domain knowledge (e.g., legal, medical) into the model itself

Decision Guidance

- If the need is about *how the model is asked*, use prompt engineering.
- If the need is about *what the model knows that changes often or is proprietary*, use RAG.
- If the need is about *how the model behaves or what skill it has*, use fine-tuning.
- If the need is about *deep, broad domain fluency* rather than a specific task, use continued pre-training.

What the Exam Tests on Customization

Expect scenario-based questions that describe a business need (cost-sensitive, needs current data, needs a new specialized skill, needs to stay unchanged but grounded) and ask you to select the correctly matched customization approach. The exam rarely asks you to perform the customization, only to correctly identify which approach fits the description.

3F. Evaluating Foundation Model Performance

Why Evaluation Is Harder for GenAI Than Traditional ML

Traditional ML models (like a spam classifier) typically have one objectively correct answer per input, so accuracy, precision, recall, and F1 score work well. Generative AI produces open-ended text, images, or audio where there can be many valid “correct” answers, making evaluation inherently more subjective and requiring different tools.

Automated Evaluation Metrics

Metric	What It Measures	Best Used For
ROUGE	Overlap between a generated summary and a human reference summary (recall-oriented)	Summarization quality
BLEU	Overlap between a generated translation and a human reference translation (precision-oriented)	Translation quality
BERTScore	Semantic similarity between generated and reference text using embeddings, rather than exact word overlap	Cases where wording differs but meaning should match
Perplexity	How “surprised” a model is by a sequence of text (lower means the model predicted the text well)	General language modeling quality, less common in applied business evaluation

Human Evaluation

Human evaluation is necessary when quality depends on subjective judgment, such as tone appropriateness, creativity, helpfulness, or harmlessness, things automated metrics struggle to capture. It is slower and more expensive than automated evaluation but provides judgment that closely mirrors real end-user experience.

Amazon Bedrock Model Evaluation

Bedrock Model Evaluation lets practitioners run automatic evaluations using built-in metrics or set up human evaluation workflows with custom criteria and reviewer teams, all within a managed AWS interface rather than building custom evaluation tooling from scratch.

Evaluation Dimensions

Beyond raw accuracy, foundation model evaluation typically considers: coherence (does the response make logical sense), relevance (does it actually address the request), harmlessness (does it avoid unsafe or offensive content), and helpfulness (does it genuinely solve the user's problem).

A/B Testing for Prompts and Models

A/B testing compares two versions of a prompt, or two different models, by running them against the same set of real or simulated requests and comparing the outcomes using either automated metrics, human ratings, or downstream business metrics (such as user satisfaction or task completion rate). This is a practical way to validate a change before fully deploying it.

3G. Business Use Cases for Foundation Models

Intelligent Document Processing and Summarization *Description:* Using an FM to extract key information from, or condense, large volumes of text such as contracts, reports, or emails. *Why FMs are suited:* FMs excel at understanding unstructured natural language and producing concise, readable summaries. *Example implementation:* A legal team uses an FM to summarize lengthy contracts into a one-page risk overview. *Key risks:* Hallucinated details, missing critical clauses, inconsistent summarization quality. *AWS implementation path:* Amazon Bedrock (model access) potentially combined with Amazon Textract (text extraction from scanned documents) feeding into the FM.

Customer-Facing Chatbots and Virtual Assistants *Description:* Conversational interfaces that answer customer questions or guide users through tasks. *Why FMs are suited:* They handle natural, varied phrasing better than rigid rules-based chatbots. *Example implementation:* A retail company deploys an FM-powered assistant that answers order status and product questions. *Key risks:* Hallucinated policy information, inconsistent tone, escalation handling for complex issues. *AWS implementation path:* Amazon Bedrock combined with Amazon Bedrock Knowledge Bases for grounding and Amazon Bedrock Guardrails for safety controls.

Code Generation and Review Assistants *Description:* FMs that help developers write, complete, or review code. *Why FMs are suited:* FMs trained on large code corpora can recognize patterns and generate syntactically correct code suggestions. *Example implementation:* A development team uses an AI coding assistant integrated into their IDE. *Key risks:* Generated code may contain subtle bugs or security vulnerabilities that still require human review. *AWS implementation path:* Amazon Q Developer or Kiro.

Knowledge Management and Enterprise Q&A *Description:* Allowing employees to ask natural-language questions and receive answers grounded in internal company documentation. *Why FMs are suited:* FMs can interpret varied phrasing of the same underlying question and synthesize an answer from retrieved content. *Example implementation:* An HR team deploys an internal tool where employees ask policy questions and receive grounded answers. *Key risks:* Outdated or

incomplete source documents leading to inaccurate answers. *AWS implementation path:* Amazon Bedrock Knowledge Bases, potentially combined with Amazon Kendra for enterprise search.

Content Creation and Marketing Automation *Description:* Using FMs to draft marketing copy, social media posts, or product descriptions. *Why FMs are suited:* FMs can rapidly generate varied, on-brand text at scale. *Example implementation:* A marketing team uses an FM with prompt templates to generate dozens of product description variants. *Key risks:* Brand voice inconsistency, factual inaccuracy in product claims, intellectual property concerns. *AWS implementation path:* Amazon Bedrock with prompt templates and Amazon Bedrock Prompt Management for version control.

Search and Information Retrieval Augmentation *Description:* Enhancing traditional keyword search with semantic understanding, so users find conceptually relevant results even without exact keyword matches. *Why FMs are suited:* Embeddings allow search systems to understand meaning rather than just matching text strings. *Example implementation:* An e-commerce site improves product search so that “warm winter jacket” also surfaces items described as “insulated parka.” *Key risks:* Irrelevant results if embeddings or indexing are poorly tuned. *AWS implementation path:* Amazon OpenSearch Service for vector storage and semantic search, often paired with Amazon Bedrock.

Data Analysis and Report Generation *Description:* Using FMs to interpret data trends and generate natural-language summaries or reports from structured data. *Why FMs are suited:* FMs can translate numerical or tabular findings into clear written narratives for non-technical stakeholders. *Example implementation:* A finance team uses an FM to generate a plain-language summary of quarterly performance from a dashboard export. *Key risks:* Misinterpretation of statistical nuance, overconfident claims not supported by the underlying data. *AWS implementation path:* Amazon Quick combined with Amazon Bedrock for narrative generation.

Translation and Localization *Description:* Using FMs to translate content into multiple languages while preserving tone and context. *Why FMs are suited:* FMs can capture contextual nuance better than older rule-based translation systems. *Example implementation:* A global company localizes its product documentation into ten languages using an FM-assisted workflow. *Key risks:* Cultural or contextual nuance lost in translation, requiring human review for high-stakes content. *AWS implementation path:* Amazon Translate for dedicated translation tasks, or Amazon Bedrock for more context-aware translation needs.

PART 4: KEY TERMS AND DEFINITIONS - DOMAIN 3

Term	Simple Definition	Everyday Analogy	Why It Matters	Common Misunderstanding
Prompt engineering	The practice of designing input text to get the desired output from a model	Giving very clear, specific directions to a new employee	It is the lowest-cost lever for improving model output	People think it requires coding skill; it does not

Term	Simple Definition	Everyday Analogy	Why It Matters	Common Misunderstanding
Zero-shot	Asking the model to do a task with no examples	Asking someone to bake a cake with no recipe, trusting their general skill	Tests the model’s general capability	Confused with “no instructions at all,” but instructions ARE given, just no examples
One-shot	Giving exactly one example before the request	Showing one sample form filled out correctly	Clarifies format with minimal token cost	Confused with few-shot; one-shot means exactly one example
Few-shot	Giving several examples before the request	Showing several solved practice problems before a test	Helps the model recognize a pattern	More examples is not always better; it raises cost and can overfit to example quirks
Chain-of-thought	Asking the model to reason step by step before answering	Asking someone to “show their work” in math class	Reduces errors in multi-step reasoning tasks	Confused with simply asking for a longer answer; it specifically requests visible reasoning steps
System prompt	Persistent instructions that govern the model’s behavior throughout a session	The standing house rules given to a babysitter before parents leave	Sets safety and behavior boundaries that should not be easily overridden	Confused with user prompt; system prompts are set by the developer, not the end user
User prompt	The specific request made by the end user in the moment	The specific question a child asks the babysitter	It is the variable, turn-by-turn input	People assume it carries the same authority as a system prompt; it does not
Retrieval-Augmented Generation (RAG)	Retrieving relevant external information and feeding it to the model before it answers	An open-book exam with a research assistant who pre-highlights the right pages	Reduces hallucination by grounding responses in real data	People think RAG retrains the model; it does not change model weights
Vector database	A database optimized to store and search embeddings by similarity	A library organized by topic similarity rather than alphabetical title	Powers fast semantic search in RAG systems	Confused with a regular database; it searches by meaning, not exact match

Term	Simple Definition	Everyday Analogy	Why It Matters	Common Misunderstanding
Embedding (vector embedding)	A numerical representation of text that captures its meaning	A GPS coordinate for an idea, placing similar ideas near each other	Enables computers to compare meaning mathematically	People think embeddings are human-readable; they are numerical vectors
Chunking	Breaking large documents into smaller pieces before embedding them	Cutting a long book into chapters before indexing it	Affects retrieval accuracy and relevance	People assume bigger chunks are always better; oversized chunks dilute relevance
Grounding	Anchoring a model's response in verified, retrieved, or referenced source material	Citing your sources in an essay instead of writing from memory alone	Directly reduces hallucination risk	Confused with fact-checking after the fact; grounding happens during generation
Agent (AI agent)	A system where an FM autonomously takes a sequence of actions, calling tools to complete a task	A personal assistant who not only answers questions but actually books the flight	Enables multi-step task automation beyond single Q&A	People think any chatbot is an agent; an agent specifically takes actions via tools
Orchestration	The coordination of multiple steps or tools an agent uses to complete a task	A conductor coordinating different sections of an orchestra	Determines how reliably a multi-step task is completed	Confused with simple sequencing; orchestration includes decision-making about which step comes next
Tool use	An agent's ability to call external functions, APIs, or systems	A mechanic reaching for the right tool from a toolbox mid-repair	Extends what an FM can do beyond generating text	People assume the FM "knows" how to use tools innately; tool use must be explicitly defined and connected
Amazon Bedrock	AWS's managed service for accessing and customizing foundation models via API	A utility company that delivers power without you building your own power plant	The central AWS platform for nearly all Domain 3 topics	Confused with SageMaker AI, which is for building custom models, not accessing pre-built FMs

Term	Simple Definition	Everyday Analogy	Why It Matters	Common Misunderstanding
Amazon Bedrock Knowledge Bases	Bedrock's managed RAG feature	A pre-built research assistant service plugged into your own filing cabinet	The primary AWS implementation of RAG tested on the exam	People think it requires building a custom vector database pipeline manually; it is managed
Amazon Bedrock Agents	Bedrock's managed feature for building autonomous, multi-step AI agents	A pre-built personal assistant framework you customize with your own tools	The AWS implementation of agentic AI for Domain 3	Confused with Strands Agents, which is the open-source development framework, not the managed Bedrock feature
Amazon Bedrock Guardrails	Bedrock's feature for enforcing safety and content policies on model input/output	A content filter and bouncer combined, screening what comes in and goes out	Central to safe, compliant GenAI deployment	People think guardrails only block bad words; they also handle topic restriction, PII masking, and grounding checks
Guardrail	A configurable rule or limit placed on what a model can produce	A guard rail on a mountain road preventing a vehicle from going off the edge	Prevents harmful, off-topic, or non-compliant outputs	Confused with a hard-coded rule in code; guardrails are configurable policies, not custom application logic
Fine-tuning (FMs)	Updating a model's weights using labeled, task-specific data	Sending an already-trained employee to a specialized workshop for one specific skill	Changes the model itself, unlike RAG or prompting	People assume fine-tuning is always necessary for customization; it is often the most expensive option, not the default
Continued pre-training	Further training a model on a large volume of unlabeled domain text	Having an employee read years of industry trade magazines to build deep familiarity	Builds broad domain fluency rather than a narrow task skill	Confused with fine-tuning; continued pre-training uses unlabeled data, not labeled task examples

Term	Simple Definition	Everyday Analogy	Why It Matters	Common Misunderstanding
Model evaluation	The process of measuring how well a model or FM-based application performs	A performance review for an employee	Determines if the model meets technical and business goals	People assume one metric (like accuracy) is sufficient; GenAI evaluation usually requires multiple metrics and/or human judgment
ROUGE score	A metric measuring overlap between a generated summary and a reference summary	Checking how much of the highlighted key sentences from a textbook chapter made it into a student's summary	Standard metric for evaluating summarization	Confused with BLEU; ROUGE is for summarization, BLEU is for translation
BERTScore	A metric measuring semantic similarity using embeddings rather than exact word overlap	Judging if two paraphrased essays mean the same thing, even with different words	Captures meaning-based similarity that ROUGE/BLEU can miss	People assume it only checks word-for-word match; it specifically captures semantic similarity
Model latency	The time it takes a model to return a response after a request	How long you wait for a reply after asking a question	Critical for real-time, user-facing applications	Confused with throughput; latency is about speed per request, not volume handled
Model throughput	The volume of requests a model can process in a given time period	How many customers a cashier can serve per hour	Critical for high-volume applications	Confused with latency; a system can have low latency per request but still low overall throughput

Term	Simple Definition	Everyday Analogy	Why It Matters	Common Misunderstanding
Context stuffing	Overloading a model's prompt/context window with excessive or irrelevant information	Cramming a desk with every file you own instead of just the ones needed for the task at hand	Increases cost and can reduce output quality and focus	People think more context is always better; irrelevant context can dilute the model's focus and raise token cost
Prompt injection	An attack where malicious instructions are inserted into a prompt or the data a model processes, to override intended behavior	Someone slipping a forged note into your mail to get you to do something you normally would not	A core AI-specific security risk tested in both Domain 3 and Domain 5	People think this only happens through direct user input; it can also occur through retrieved documents or external data
Inference parameters	Configurable settings (like temperature, max tokens) that shape a model's output at the time of generation	The settings dial on an oven: same recipe, different result depending on temperature and time	Allow practitioners to tune output behavior without retraining	People think these change the model permanently; they only affect a single inference call
Model provider	The organization or entity that created and trained a given foundation model	The brand or manufacturer of an appliance	Affects licensing, capability, and cost	People assume all models accessible via Bedrock are made by Amazon; Bedrock hosts multiple third-party providers as well as Amazon's own Nova models
Semantic search	Search based on meaning and context rather than exact keyword matching	A librarian who understands what you mean, not just the exact words you used	Powers more accurate RAG retrieval and enterprise search	Confused with simple keyword search; semantic search relies on embeddings, not text matching

Term	Simple Definition	Everyday Analogy	Why It Matters	Common Misunderstanding
Hybrid search	Search that combines keyword (exact match) search with semantic (meaning-based) search	A librarian who checks both the exact title AND the general topic of your request	Often produces more accurate retrieval results than either method alone	People assume hybrid search is always semantic search alone; it specifically blends both approaches

PART 5: COMPARISON TABLES - DOMAIN 3

Table 1: Prompt Engineering vs. RAG vs. Fine-Tuning vs. Continued Pre-Training

Approach	Definition	Data Needed	Model Changed?	Cost	Best For	AWS Support
Prompt engineering	Designing input text to guide model behavior	None	No	Lowest	Quick adjustments, simple tasks	Amazon Bedrock Prompt Management
RAG	Retrieving external data and adding it to the prompt at query time	Existing documents (no labeling required)	No	Low to moderate	Grounding in current or proprietary information	Amazon Bedrock Knowledge Bases
Fine-tuning	Updating model weights using labeled, task-specific examples	Labeled, task-specific dataset	Yes	Moderate to high	Teaching new skills, styles, or specialized tasks	Amazon Bedrock Fine-Tuning
Continued pre-training	Further training the model using large volumes of unlabeled domain text	Large unlabeled domain corpus	Yes	Highest	Broad domain fluency (e.g., legal, medical)	Amazon Bedrock continued pre-training

Table 2: Zero-Shot vs. One-Shot vs. Few-Shot Prompting

Factor	Zero-Shot	One-Shot	Few-Shot
Examples provided	None	Exactly one	Multiple (typically 2-5)
Token usage	Lowest	Low	Higher
Best for	Simple, well-known tasks	Slightly ambiguous formatting needs	Nuanced or pattern-heavy tasks
Risk	May misunderstand desired format	Single example may not cover edge cases	Higher cost; inconsistent examples can confuse the model

Table 3: Amazon Bedrock Knowledge Bases vs. Amazon Kendra

Factor	Amazon Bedrock Knowledge Bases	Amazon Kendra
Purpose	Managed RAG pipeline that grounds FM responses in company data	Intelligent enterprise search service with natural-language query
RAG-native?	Yes, purpose-built specifically to feed retrieved context into an FM	Not RAG-native by default; built for direct search results, though it can be paired with an FM
Typical use case	Powering a GenAI chatbot that must answer using company documents	Enterprise-wide document search across many systems for employees
When to choose each	Choose Knowledge Bases when the end goal is an FM-generated, conversational answer grounded in data	Choose Kendra when the end goal is precise, ranked search results across diverse enterprise data sources

Table 4: Automated Evaluation vs. Human Evaluation

Factor	Automated Evaluation	Human Evaluation
Speed	Fast, scalable	Slow, limited by reviewer availability
Cost	Lower	Higher
Objectivity	Consistent but limited to what the metric measures	Captures nuance and subjective quality, but can vary between reviewers
Best for	Large-scale, repeatable testing (e.g., regression testing after a prompt change)	Tone, safety, helpfulness, and other subjective qualities

Factor	Automated Evaluation	Human Evaluation
AWS tool	Amazon Bedrock Model Evaluation (automatic mode)	Amazon Bedrock Model Evaluation (human evaluation workflows); Amazon A2I for human review of low-confidence outputs

Table 5: RAG vs. Long Context Window

Factor	RAG	Long Context Window
What it solves	Grounding responses in large or frequently updated external knowledge	Allowing a single prompt to include a large amount of information directly
Cost	Lower per-query cost since only relevant chunks are retrieved and included	Higher cost since the entire context is processed on every call
Accuracy	Generally higher for large knowledge bases, since only the most relevant content is surfaced	Can degrade if irrelevant information is mixed in with relevant information (“needle in a haystack” risk)
Freshness	Easy to keep current by updating the underlying document source	Requires re-supplying updated content in every prompt
When to use	Large, frequently changing, or very large knowledge bases	Smaller, well-defined documents that fit comfortably within the context limit and do not change often

Table 6: Amazon Bedrock Agents vs. Traditional API Integration

Factor	Amazon Bedrock Agents	Traditional API Integration
Autonomy	The FM decides which tools/APIs to call and in what order, based on reasoning	Calls are explicitly hard-coded by a developer in a fixed sequence
Multi-step tasks	Naturally suited to dynamic, multi-step task completion that can vary by request	Requires the developer to anticipate and code every possible path in advance
Setup complexity	Lower long-term complexity for varied or evolving tasks, since the FM handles reasoning	Can be simpler for a single, fixed, well-defined task, but rigid

Factor	Amazon Bedrock Agents	Traditional API Integration
Use case	Customer requests that require judgment and a variable sequence of steps (e.g., “investigate and resolve this issue”)	Predictable, repetitive integrations with a known, unchanging workflow (e.g., a fixed nightly data sync)

PART 6: 25 DOMAIN 3 PRACTICE QUESTIONS

Beginner (8 Questions)

1. (Multiple Choice) A prompt engineer asks a foundation model to classify customer feedback as positive, negative, or neutral, providing only the instruction and no examples. Which prompting technique is this?

A. Few-shot prompting B. Zero-shot prompting C. Chain-of-thought prompting D. Fine-tuning

Answer: B. No examples are provided, only an instruction, which defines zero-shot prompting. Few-shot would require multiple examples (A). Chain-of-thought asks for step-by-step reasoning, not classification format (C). Fine-tuning changes model weights and is unrelated to prompt structure (D).

2. (Multiple Choice) What is the primary purpose of Retrieval-Augmented Generation (RAG)?

A. To permanently update a model’s weights with new information B. To retrieve relevant external information and provide it to the model at the time of the request C. To compress a large model into a smaller, faster one D. To translate prompts into multiple languages automatically

A is incorrect because RAG does not change model weights. **Answer: B.** RAG retrieves relevant data and augments the prompt with it before generation. C describes model distillation. D is unrelated to RAG’s function.

3. (Multiple Choice) Which AWS service is the managed implementation of RAG that connects a foundation model to a company’s documents stored in Amazon S3?

A. Amazon Bedrock Guardrails B. Amazon Bedrock Knowledge Bases C. Amazon Bedrock Model Evaluation D. Amazon SageMaker Clarify

Answer: B. Amazon Bedrock Knowledge Bases is purpose-built for managed RAG against data sources like Amazon S3. Guardrails (A) handles safety policies, not retrieval. Model Evaluation (C) assesses performance. SageMaker Clarify (D) detects bias, an unrelated Domain 4 tool.

4. (Multiple Choice) Which inference parameter controls how creative or random a foundation model’s output is?

A. Context window B. Temperature C. Token limit D. Latency

Answer: B. Temperature controls the randomness/creativity of output. Context window (A) controls how much information the model considers. Token limit (C) controls output length. Latency (D) measures response speed, not output style.

5. (Multiple Choice) A company wants to update a foundation model’s weights using its own labeled, task-specific dataset to teach it a specialized skill. Which approach is this?

A. Prompt engineering B. RAG C. Fine-tuning D. Semantic search

Answer: C. Fine-tuning specifically updates model weights using labeled data. Prompt engineering (A) and RAG (B) do not change the model. Semantic search (D) is a retrieval technique, not a customization method.

6. (Multiple Choice) Which metric is most appropriate for evaluating the quality of a machine-generated text summary against a human-written reference summary?

A. BLEU B. ROUGE C. Latency D. F1 score

Answer: B. ROUGE is specifically designed for summarization evaluation. BLEU (A) is for translation. Latency (C) measures speed, not quality. F1 score (D) is a classification metric, not suited to open-ended summarization comparison.

7. (Multiple Choice) What does “chunking” refer to in the context of a RAG system?

A. Reducing a foundation model’s size for faster inference B. Breaking large documents into smaller pieces before embedding and storage C. Filtering toxic content from model output D. Splitting a prompt into system and user components

Answer: B. Chunking breaks documents into smaller, retrievable pieces. A describes model distillation. C describes guardrail functionality. D describes prompt structure, unrelated to chunking.

8. (Multiple Choice) Which AWS feature allows a foundation model to autonomously call external tools and APIs to complete a multi-step task?

A. Amazon Bedrock Knowledge Bases B. Amazon Bedrock Agents C. Amazon Bedrock Guardrails D. Amazon Bedrock Model Evaluation

Answer: B. Bedrock Agents is built specifically for autonomous, multi-step, tool-calling behavior. Knowledge Bases (A) handles retrieval, not action-taking. Guardrails (C) enforces safety policy. Model Evaluation (D) assesses performance.

Intermediate (12 Questions)

9. (Multiple Choice) A company’s foundation-model-powered chatbot needs to answer questions using internal policy documents that change weekly, without retraining the model. Which approach is MOST appropriate?

A. Continued pre-training B. Fine-tuning C. RAG D. Model distillation

Answer: C. RAG is ideal for frequently changing information because updating a document source is far simpler than retraining. Continued pre-training (A) and fine-tuning (B) both require retraining cycles unsuited to weekly changes. Distillation (D) creates a smaller model and does not address information freshness.

10. (Multiple Choice) Which scenario BEST describes prompt hijacking?

A. A malicious actor inserts hidden instructions into a document that a RAG system later retrieves
B. A user crafts input that overrides the application's intended behavior to make the model perform an unintended action
C. A developer forgets to specify an output format in the prompt
D. A model produces inconsistent answers to the same prompt on different occasions

Answer: B. Prompt hijacking is a user input directly overriding intended behavior. A describes prompt poisoning (injected via external data). C is a prompt design mistake, not an attack. D describes nondeterminism, not hijacking.

11. (Multiple Choice) A company needs a foundation model to gain deep, broad familiarity with legal terminology across an enormous volume of unlabeled legal text, without targeting one specific task. Which technique is MOST appropriate?

A. Few-shot prompting B. RAG C. Continued pre-training D. Chain-of-thought prompting

Answer: C. Continued pre-training uses large volumes of unlabeled domain text to build broad fluency. Few-shot prompting (A) and chain-of-thought (D) are prompting techniques, not training approaches. RAG (B) retrieves specific documents at query time rather than building broad model knowledge.

12. (Multiple Choice) Which factor would MOST influence a decision between Amazon OpenSearch Service and Amazon Neptune for storing vector embeddings in a RAG application?

A. Whether the application primarily needs flexible search/analytics versus highly connected, relationship-based data
B. Whether the application is hosted in a single AWS Region
C. Whether the company prefers open-source software
D. Whether the data needs to be encrypted at rest

Answer: A. OpenSearch is suited for flexible search/analytics use cases, while Neptune is optimized for highly connected, graph-like data with vector search support. B, C, and D are not meaningful differentiators between these two specific services for this decision.

13. (Multiple Choice) A development team wants to track changes made to a production prompt over time and roll back to an earlier version if needed. Which AWS capability supports this?

A. Amazon Bedrock Guardrails B. Amazon Bedrock Prompt Management C. Amazon SageMaker Model Cards D. AWS CloudTrail

Answer: B. Amazon Bedrock Prompt Management is specifically designed for prompt versioning. Guardrails (A) enforces safety policy. Model Cards (C) document model behavior, not prompt

versions. CloudTrail (D) logs account activity generally, not prompt-specific versioning.

14. (Multiple Choice, Multiple Response) Which TWO of the following are valid reasons to choose fine-tuning over RAG for a given use case? (Select TWO.)

A. The underlying knowledge changes daily B. The model needs to learn an entirely new specialized writing style or task behavior C. The company has a well-labeled, task-specific training dataset available D. The company wants the lowest possible implementation cost E. The company wants to avoid touching the model's weights at all

Answer: B and C. Fine-tuning is appropriate when the model needs to learn new behavior or style (B) and when labeled task-specific data is available (C). A describes a case better suited to RAG. D and E both describe reasons to AVOID fine-tuning, not adopt it.

15. (Multiple Choice) A company wants its FM-powered assistant to refuse to discuss competitor pricing and to redact any customer Social Security numbers that might appear in a response. Which AWS feature should it use?

A. Amazon Bedrock Knowledge Bases B. Amazon Bedrock Agents C. Amazon Bedrock Guardrails D. Amazon Bedrock Model Evaluation

Answer: C. Guardrails handles topic restriction and PII redaction. Knowledge Bases (A) is for retrieval/grounding. Agents (B) is for multi-step task execution. Model Evaluation (D) assesses performance, not policy enforcement.

16. (Multiple Choice) Which of the following BEST explains why few-shot prompting can increase cost compared to zero-shot prompting?

A. Few-shot prompting requires a separate fine-tuning job B. Few-shot prompting includes additional example text in the prompt, increasing token usage C. Few-shot prompting requires a vector database D. Few-shot prompting always uses a larger, more expensive model

Answer: B. The added examples consume more tokens, which directly increases cost under token-based pricing. A, C, and D are all unrelated to how few-shot prompting actually works.

17. (Multiple Choice) A company is evaluating an FM-powered translation tool and wants an automated metric that compares the machine translation against a human reference translation. Which metric should it use?

A. ROUGE B. BLEU C. F1 score D. Perplexity

Answer: B. BLEU is purpose-built for translation evaluation. ROUGE (A) targets summarization. F1 score (C) is a classification metric. Perplexity (D) measures general language modeling fit, not translation accuracy against a reference.

18. (Ordering) Place the following steps of a RAG pipeline in the correct order, from first to last.

A. The foundation model generates a response using the retrieved content and the original query
B. The user’s query is converted into a vector embedding
C. The most relevant document chunks are retrieved from the vector database
D. The retrieved chunks are added to the prompt alongside the original query

Answer: B, C, D, A. The query is first embedded (B), relevant chunks are retrieved (C), those chunks are added to the prompt (D), and finally the model generates a grounded response (A).

19. (Multiple Choice) A company has deployed an FM-based customer service agent and wants to measure whether it is delivering real business value. Which metric is MOST directly aligned with this goal?

A. BLEU score B. Task completion rate C. Token-based pricing rate D. Context window size

Answer: B. Task completion rate directly reflects whether the agent is successfully resolving user goals, a business-aligned outcome metric. BLEU (A) is a technical translation metric unrelated to this use case. Token pricing (C) and context window size (D) are technical/cost characteristics, not business outcome measures.

20. (Multiple Choice) Which statement correctly distinguishes Amazon Bedrock Knowledge Bases from Amazon Kendra?

A. Knowledge Bases is for building enterprise search portals; Kendra is for grounding FM responses
B. Knowledge Bases is purpose-built to feed retrieved content into an FM response; Kendra is a broader intelligent enterprise search service
C. Knowledge Bases and Kendra perform identical functions and can be used interchangeably in all cases
D. Kendra requires a foundation model to function; Knowledge Bases does not

Answer: B. This correctly distinguishes the RAG-native purpose of Knowledge Bases from Kendra’s broader enterprise search role. A reverses their purposes. C is incorrect since their primary design goals differ. D is incorrect; Kendra functions as a search service independent of an FM.

Exam-Ready (5 Questions)

21. (Multiple Choice) A company needs to deploy a foundation model for a narrow, high-volume, repetitive classification task and wants to minimize both cost and response latency while still meeting accuracy requirements. Which approach reflects the BEST practice described in this module?

A. Always select the largest, most capable foundation model available
B. Select the smallest, most cost-effective model that reliably meets the accuracy requirement for the task
C. Use continued pre-training on the largest available model before deployment
D. Use chain-of-thought prompting on the largest model to maximize accuracy regardless of cost

Answer: B. This reflects the “good enough” principle: choose the smallest, most cost-effective model that still meets requirements, rather than defaulting to the largest model (A, C, D), each of which increases cost and/or latency unnecessarily for a narrow, repetitive task.

22. (Multiple Choice, Multiple Response) A company is designing a foundation-model-based application and wants to reduce hallucination risk. Which TWO approaches would directly help achieve this goal? (Select TWO.)

A. Using RAG to ground responses in verified source documents B. Increasing the temperature inference parameter to maximize creativity C. Implementing Amazon Bedrock Guardrails grounding checks D. Removing all context from the prompt to simplify the request E. Switching to a model with a smaller context window

Answer: A and C. RAG grounds responses in verified data (A), and Guardrails grounding checks verify that responses are supported by source material (C), both directly reducing hallucination. Increasing temperature (B) increases randomness and can worsen hallucination risk. Removing context (D) removes grounding information. Context window size (E) does not directly address hallucination.

23. (Matching) Match each AWS Bedrock feature on the left with its primary purpose on the right.

Features: 1. Amazon Bedrock Knowledge Bases 2. Amazon Bedrock Agents 3. Amazon Bedrock Guardrails 4. Amazon Bedrock Model Evaluation

Purposes: A. Enforces content safety and topic policies on model input/output B. Implements managed RAG by connecting an FM to company data C. Assesses and compares foundation model performance using automated or human methods D. Enables an FM to autonomously call tools/APIs to complete multi-step tasks

Answer: 1-B, 2-D, 3-A, 4-C. Knowledge Bases implements RAG (B); Agents enables autonomous, tool-using multi-step behavior (D); Guardrails enforces safety/content policy (A); Model Evaluation assesses performance (C).

24. (Multiple Choice) A company's foundation model occasionally produces well-written but factually incorrect statements presented with high confidence. The company wants an automated way to detect when this is likely to happen before the response reaches the end user. Which approach is MOST directly relevant?

A. Increasing the model's temperature setting B. Using confidence scoring alongside output validation as part of a grounding strategy C. Switching to single-shot prompting exclusively D. Increasing the context window size without changing retrieval behavior

Answer: B. Confidence scoring and output validation are specifically used to detect and flag potentially unreliable, hallucinated outputs before they reach the user. Increasing temperature (A) would likely worsen the problem by increasing randomness. Single-shot prompting (C) and a larger context window alone (D) do not directly address hallucination detection.

25. (Multiple Choice) A company must choose between fine-tuning a foundation model and using in-context learning (such as few-shot prompting) to adapt the model's behavior for a moderately

specialized task. The company has only a small number of labeled examples and needs to deploy a working solution within the week. Which approach is MOST appropriate given these constraints?

A. Fine-tuning, because it always produces superior results regardless of data volume B. In-context learning (few-shot prompting), because it requires no separate training process and can be implemented immediately with the available examples C. Continued pre-training, because it is the fastest customization method available D. Model distillation, because it eliminates the need for any examples at all

Answer: B. With only a small number of examples and a tight timeline, in-context learning through few-shot prompting can be deployed immediately without a training cycle. Fine-tuning (A) typically requires more data and time than is available here. Continued pre-training (C) is the slowest, not the fastest, customization method. Model distillation (D) serves a different purpose (creating a smaller model from a larger one's outputs) and does not fit this scenario.

PART 7: 30 DOMAIN 3 FLASHCARDS

Terminology Cards (8)

Card 1 - Terminology Front: What is Retrieval-Augmented Generation (RAG)? Back: A technique where relevant external information is retrieved and added to a model's prompt before it generates a response, grounding the output in real data without changing model weights. Exam tip: RAG does NOT retrain the model.

Card 2 - Terminology Front: What is an embedding (vector embedding)? Back: A numerical representation of text that captures its meaning, allowing similar meanings to be mathematically compared. Exam tip: embeddings power semantic search and RAG retrieval.

Card 3 - Terminology Front: What is a vector database? Back: A database optimized to store embeddings and perform fast similarity searches across them. Exam tip: AWS options include Amazon OpenSearch Service, Amazon Aurora, Amazon Neptune, and Amazon RDS for PostgreSQL with pgvector.

Card 4 - Terminology Front: What are Amazon Bedrock Guardrails? Back: A Bedrock feature that enforces safety and policy limits on model input/output, including content filters, topic restrictions, PII masking, and grounding checks. Exam tip: Guardrails is the answer when a scenario asks about restricting topics or redacting sensitive data.

Card 5 - Terminology Front: What is an AI agent in the context of Amazon Bedrock? Back: A system where a foundation model autonomously takes a sequence of actions, calling external tools or APIs, to complete a multi-step task. Exam tip: Amazon Bedrock Agents is the managed AWS feature for this.

Card 6 - Terminology Front: What is chunking? Back: The process of breaking large documents into smaller pieces before they are embedded and stored in a vector database, to improve retrieval relevance. Exam tip: chunk size affects retrieval quality; too large dilutes relevance, too small loses context.

Card 7 - Terminology Front: What is grounding? Back: Anchoring a model's response in verified, retrieved, or referenced source material to reduce hallucination. Exam tip: RAG and Bedrock Guardrails grounding checks are both grounding techniques.

Card 8 - Terminology Front: What is prompt injection? Back: An attack where malicious instructions are inserted into a prompt or the data a model processes, to override intended behavior. Exam tip: this can occur through direct user input or through external/retrieved data (the latter is sometimes specifically called prompt poisoning).

Scenario Cards (10)

Card 9 - Scenario Front: A company wants to connect a foundation model to an internal database of product manuals without retraining the model. What should they use? Back: Retrieval-Augmented Generation (RAG), implemented via Amazon Bedrock Knowledge Bases.

Card 10 - Scenario Front: A team needs the model to complete a multi-step task: look up an order, check inventory, and issue a refund. What AWS feature fits this need? Back: Amazon Bedrock Agents, which allows the FM to autonomously call tools/APIs across multiple steps.

Card 11 - Scenario Front: A company wants to prevent its FM-powered chatbot from ever discussing competitor pricing. What should they configure? Back: Amazon Bedrock Guardrails, using topic restriction policies.

Card 12 - Scenario Front: A company has a small, well-labeled dataset and needs the model to learn a brand-new specialized writing style. What customization approach fits best? Back: Fine-tuning, since it is designed to teach new task-specific behavior using labeled data.

Card 13 - Scenario Front: A company needs broad legal domain fluency built into the model using a huge volume of unlabeled legal text. What approach fits best? Back: Continued pre-training.

Card 14 - Scenario Front: A company wants the fastest, lowest-cost way to slightly adjust a model's tone and output format with no available training data. What should they do? Back: Prompt engineering (e.g., adjusting instructions, adding negative prompts, or using role/persona prompting).

Card 15 - Scenario Front: A company wants to automatically evaluate machine-generated summaries against human reference summaries. What metric should they use? Back: ROUGE.

Card 16 - Scenario Front: A company wants to automatically evaluate machine-generated translations against human reference translations. What metric should they use? Back: BLEU.

Card 17 - Scenario Front: A company wants to measure semantic similarity between a generated response and a reference answer, even if the wording differs. What metric should they use? Back: BERTScore.

Card 18 - Scenario Front: A company wants to track and roll back changes to its production prompts over time. What AWS capability supports this? Back: Amazon Bedrock Prompt Management.

Comparison Cards (7)

Card 19 - Comparison Front: What is the difference between RAG and fine-tuning? Back: RAG retrieves external information at query time without changing the model's weights; fine-tuning permanently updates the model's weights using labeled training data.

Card 20 - Comparison Front: What is the difference between zero-shot and few-shot prompting? Back: Zero-shot provides no examples, only instructions; few-shot provides multiple examples to

demonstrate the desired pattern before the actual request.

Card 21 - Comparison Front: What is the difference between fine-tuning and continued pre-training? Back: Fine-tuning uses labeled, task-specific data to adapt a model to a specific task; continued pre-training uses large volumes of unlabeled domain text to build broader domain knowledge.

Card 22 - Comparison Front: What is the difference between prompt hijacking and prompt poisoning? Back: Prompt hijacking occurs when user input directly overrides intended model behavior; prompt poisoning occurs when malicious instructions are injected indirectly through external data the model processes, such as a retrieved document.

Card 23 - Comparison Front: What is the difference between Amazon Bedrock Knowledge Bases and Amazon Kendra? Back: Knowledge Bases is purpose-built for RAG, feeding retrieved content directly into an FM's response; Kendra is a broader intelligent enterprise search service that returns ranked search results and is not RAG-native by default.

Card 24 - Comparison Front: What is the difference between model latency and model throughput? Back: Latency measures how long a single request takes to receive a response; throughput measures how many requests a model can process over a given period of time.

Card 25 - Comparison Front: What is the difference between automated evaluation and human evaluation of foundation models? Back: Automated evaluation uses metrics like ROUGE, BLEU, and BERTScore for fast, scalable, repeatable testing; human evaluation uses human reviewers to judge subjective qualities like tone, helpfulness, and safety that automated metrics cannot fully capture.

Bedrock Feature Identification Cards (5)

Card 26 - Bedrock Feature Front: Which Bedrock feature implements managed RAG by connecting an FM to data in Amazon S3? Back: Amazon Bedrock Knowledge Bases.

Card 27 - Bedrock Feature Front: Which Bedrock feature enables an FM to autonomously call tools and APIs to complete multi-step tasks? Back: Amazon Bedrock Agents.

Card 28 - Bedrock Feature Front: Which Bedrock feature enforces content filters, topic restrictions, PII masking, and grounding checks? Back: Amazon Bedrock Guardrails.

Card 29 - Bedrock Feature Front: Which Bedrock feature supports both automated and human-based assessment of model output quality? Back: Amazon Bedrock Model Evaluation.

Card 30 - Bedrock Feature Front: Which Bedrock feature allows a company to update model weights using their own labeled, task-specific dataset? Back: Amazon Bedrock Fine-Tuning.

PART 8: DOMAIN 3 ONE-PAGE CHEAT SHEET

The Customization Decision Tree

START: Do you need to change how the model is asked to do something,
with no new data and no model changes?

|

YES --> Use PROMPT ENGINEERING (zero-shot, few-shot, chain-of-thought, etc.)


```

|
NO, continue:
|
Does the model need access to current, proprietary, or frequently
changing information it was not trained on?
|
YES --> Use RAG (Amazon Bedrock Knowledge Bases)
|
NO, continue:
|
Does the model need to learn a brand-new skill, style, or specialized
task behavior, and do you have labeled examples?
|
YES --> Use FINE-TUNING
|
NO, continue:
|
Does the model need deep, broad fluency in a domain (not one task),
built from large volumes of unlabeled text?
|
YES --> Use CONTINUED PRE-TRAINING

```

Top 10 Domain 3 Concepts to Understand

1. RAG grounds a model in external data WITHOUT changing its weights.
2. Fine-tuning DOES change model weights and requires labeled data.
3. Prompt engineering is the fastest, cheapest customization lever.
4. Zero-shot = no examples; one-shot = one example; few-shot = multiple examples.
5. Chain-of-thought prompting asks the model to reason step by step.
6. ROUGE = summarization metric; BLEU = translation metric.
7. Vector databases store embeddings for similarity-based (semantic) search.
8. Amazon Bedrock Agents enables autonomous, multi-step, tool-using behavior.
9. Amazon Bedrock Guardrails enforces safety policy, topic limits, and PII masking.
10. Continued pre-training builds broad domain knowledge; fine-tuning builds task-specific skill.

Bedrock Features at a Glance

Feature	One-Line Purpose
Amazon Bedrock (core)	API access to foundation models from multiple providers
Knowledge Bases	Managed RAG: grounds FM responses in company data
Agents	Autonomous, multi-step, tool-using AI agents
Guardrails	Safety policy enforcement: content filters, topic limits, PII masking, grounding checks
Model Evaluation	Automated and human evaluation of FM performance

Feature	One-Line Purpose
Fine-Tuning	Customizes model weights using labeled, task-specific data
Prompt Management	Versioning and management of production prompts

Essential Domain 3 Vocabulary

RAG, embedding, vector database, chunking, grounding, agent, orchestration, tool use, guardrail, fine-tuning, continued pre-training, zero-shot, few-shot, chain-of-thought, ROUGE, BLEU, BERTScore, prompt injection, prompt poisoning, prompt hijacking, jailbreaking, temperature, context window, latency, throughput.

5 Domain 3 Exam Watch-Outs

1. RAG does not retrain or change the model; it only changes what the model sees at query time.
2. ROUGE is for summarization; BLEU is for translation. Do not swap them.
3. Prompt hijacking (direct override) is different from prompt poisoning (injected through external/retrieved data).
4. A bigger or more general model is not automatically the “MOST appropriate” exam answer; cost-effectiveness and fit to the task usually win.
5. Continued pre-training (unlabeled domain text, broad knowledge) is different from fine-tuning (labeled task data, specific skill).

“If the Question Says...” Exam Clue Pairs

If the question says...	Think...
“Without retraining the model” / “ground responses in company data”	RAG / Amazon Bedrock Knowledge Bases
“Teach the model a new specialized skill” / “labeled examples available”	Fine-tuning
“Broad domain knowledge” / “large volume of unlabeled text”	Continued pre-training
“No examples, just instructions”	Zero-shot prompting
“Several examples before the request”	Few-shot prompting
“Show your reasoning step by step”	Chain-of-thought prompting
“Restrict topics” / “redact PII” / “filter harmful content”	Amazon Bedrock Guardrails
“Multi-step task” / “calls an API” / “takes autonomous action”	Amazon Bedrock Agents
“Compare summaries to a reference”	ROUGE
“Compare translations to a reference”	BLEU
“Semantic similarity despite different wording”	BERTScore
“Track and roll back prompt versions”	Amazon Bedrock Prompt Management

PART 9: DOMAIN 3 SUMMARY AND REVIEW PLAN

5 Key Domain 3 Takeaways

1. **The customization decision framework is the heart of Domain 3.** Nearly every scenario question can be solved by asking: does this need prompt engineering, RAG, fine-tuning, or continued pre-training? Knowing the decision framework cold will resolve a large share of Domain 3 questions.
2. **RAG and fine-tuning are not interchangeable.** RAG grounds a model in external data without touching its weights; fine-tuning changes the model itself using labeled data. Confusing these two is the single most common Domain 3 mistake.
3. **Prompt engineering techniques have precise definitions.** Zero-shot, one-shot, few-shot, and chain-of-thought are each defined by a specific, recognizable pattern (number of examples, or presence of step-by-step reasoning).
4. **Amazon Bedrock's feature set maps directly to task statements.** Knowledge Bases (RAG), Agents (multi-step action), Guardrails (safety/policy), Model Evaluation (assessment), and Fine-Tuning (customization) each correspond to a specific recurring exam pattern.
5. **Evaluation requires the right metric for the right task.** ROUGE for summarization, BLEU for translation, BERTScore for semantic similarity, and human evaluation or LLM-as-a-judge for subjective qualities that automated metrics cannot capture.

Remediation Plan for Scores Below 60%

If your practice question results for Domain 3 fall below 60%, follow this remediation sequence before moving forward:

1. Re-read Part 3A (Prompt Engineering) and Part 3B (RAG) in full; these two sections account for the largest share of Domain 3 exam content.
2. Redo the Comparison Table in Part 5, Table 1, from memory (Prompt Engineering vs. RAG vs. Fine-Tuning vs. Continued Pre-Training), writing out each column without looking, then check your answers.
3. Re-attempt all 25 practice questions in Part 6, this time reading the rationale for every question, including the ones you answered correctly, to confirm your reasoning matches the explanation.
4. Drill the 30 flashcards in Part 7 in full at least twice, focusing extra repetitions on the Comparison Cards (19-25), since comparison-based confusion is the most common source of missed points.
5. Review the "If the Question Says..." clue table in Part 8 and practice matching each clue phrase to its concept without looking at the answer column.

3-Day Focused Domain 3 Review Schedule

Day 1 (Foundations and Prompting): - Review Part 1 (Introduction) and Part 2 (Task Statement Breakdown). - Deep review of Part 3A (Prompt Engineering) and the prompt engineering comparison table. - Self-test using Flashcards 1-8 (Terminology) and 19-22 (Comparison).

Day 2 (RAG, Bedrock Features, and Customization): - Deep review of Part 3B (RAG), Part 3C (Bedrock Deep Dive), and Part 3E (Customization Spectrum). - Self-test using Flashcards 9-18 (Scenario) and 26-30 (Bedrock Feature Identification). - Re-attempt Comparison Tables 1, 3, and 6 from Part 5 without looking at the answers first.

Day 3 (Model Selection, Evaluation, and Practice): - Deep review of Part 3D (Model Selection) and Part 3F (Evaluation). - Complete all 25 practice questions in Part 6 under timed conditions (aim for under 30 minutes). - Review the One-Page Cheat Sheet (Part 8) in full as a final pass.

How Domain 3 Connects to Domain 4 and Domain 5

Domain 3 does not exist in isolation. The foundation model applications covered here become the subject matter of the next two domains:

- **Connection to Domain 4 (Guidelines for Responsible AI):** Once you know how to build and customize FM-based applications, Domain 4 asks whether you are doing so responsibly. The same Amazon Bedrock Guardrails introduced here for safety filtering reappears in Domain 4 as a tool for enforcing fairness and safety. Evaluation concepts from Part 3F also connect directly to Domain 4's focus on detecting bias and ensuring trustworthy outputs.
- **Connection to Domain 5 (Security, Compliance, and Governance):** The risks introduced in Part 3A, such as prompt injection, prompt poisoning, and jailbreaking, are explored in much greater depth in Domain 5 as AI-specific security threats. Similarly, the grounding and hallucination-reduction techniques from Part 3B and Part 3F (RAG grounding, output validation, confidence scoring) are the same techniques Domain 5 covers under "hallucination detection and grounding."

Mastering Domain 3 thoroughly will make both Domain 4 and Domain 5 significantly easier to learn, since many of the same AWS tools and underlying concepts reappear, applied through a responsibility and security lens rather than a pure application-design lens.

End of Domain 3 Study Module - Task 4 of 15 Next task: Task 5 - Domain 4 Study Module: Guidelines for Responsible AI